LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – DASAR KONSEP

NAMA : Jeremy Marcello Waani

NIM : 2311110003

KELAS : PS1DS-05

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (PBO) adalah paradigma pengembangan perangkat lunak berbasis objek yang merepresentasikan entitas nyata melalui penggabungan struktur data (state/attribute) dan fungsionalitas (behavior/method) dalam satu kesatuan modular. Paradigma ini dibangun di atas empat pilar utama, yaitu abstraksi untuk menyaring informasi esensial dan mengabaikan detail insidental, enkapsulasi (data hiding) untuk memproteksi integritas data internal menggunakan access modifier (seperti private dan public), pewarisan (inheritance) untuk penggunaan kembali kode, serta polimorfisme untuk menyediakan antarmuka seragam bagi berbagai bentuk perilaku. Dalam implementasi Java, class bertindak sebagai cetak biru (blueprint) dan object sebagai hasil instansiasi fisik di memori, yang dapat tersusun atas instance field milik tiap objek secara individual maupun class field (menggunakan kata kunci static) yang dipakai bersama secara global. Selain itu, Java mengelola eksekusi method melalui konstruktor yang dapat memanfaatkan kata kunci referensi this untuk resolusi ambiguitas penamaan, serta menerapkan mekanisme pemanggilan argumen berupa pass-by-value untuk tipe data primitif dan pass-by-reference (penyalinan alamat referensi memori Heap) untuk tipe data objek dan array.

B. Guided

1. Guided 1: Abstraksi Objek

```
package guided;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing oren = new Kucing();
        oren.nama = "Cimot Gendot";
        oren.ras = "Angora";
        oren.bersuara();
    }
}
```

```
package guided;

public class Kucing {
    String nama;
    String ras;

    void bersuara() {
        System.out.println(nama + " " + ras + " Meow");
    }

}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-03-konsep-dasar ---
Cimot Gendot Angora Meow
```

2. Guided 2: Enkapsulasi dan Data Hiding

```
package guided;

public class Rekening {
  private int saldo = 0;

   public void tambahsaldo (int jumlah) {
      saldo += jumlah;
      System.out.println("Saldo berhasil ditambahkan: " + saldo);
   }

   public void tampilkansaldo(){
      System.out.println("Saldo saat ini: " + saldo);
   }
}
```

```
package guided;

public class Enkapsulasi {

   public static void main(String[] args) {
      Rekening rek = new Rekening();
      rek.tambahsaldo(300000000);
      rek.tampilkansaldo();
   }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-03-konsep-dasar ---
Saldo berhasil ditambahkan: 300000000
Saldo saat ini: 300000000
```

3. Guided 3: Class Field dan Instance Field

```
package guided;

public class Circle01 {
   public static final double PI = 3.14;

   public static double radiansToDegrees(double rads) {
      return rads * (180 / PI);
   }

   public double r;
```

```
    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }

}
```

4. Guided 4: Parameter Passing

```
package guided;

public class TestPass {
    int i,j;

    TestPass(int a,int b) {
        i = a;
        j = b;
    }
    void calculate(int m,int n) {
        m = m*10;
        n = n/2;
    }
 //passed by reference dengan berupa tipe data class
    void calculate(TestPass e) {
        e.i = e.i*10;
        e.j = e.j/2;
 }
}
```

```
package guided;

public class PassedByValue {
    public static void main(String[] args) {
  int x,y;
        TestPass z;
        z = new TestPass(50,100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        //passed by value
```

```
        z.calculate(x,y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        //passed by reference
        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-03-konsep-dasar ---
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
```

5. Guided 5: Penggunaan Keyword this pada Method

```
package guided;

public class Lagu {
    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }
    public void cetakKeLayar(){
        if(judul==null && pencipta==null) return;
        System.out.println("Judul : " + judul +", pencipta : " + pencipta);
    }
```

```
    static class DemoLagu {
        public static void main(String[] args){
            Lagu a = new Lagu();
            a.IsiParam("God Will Make A Way", "Don Moen");
            a.cetakKeLayar();
        }
    }
}
        int a = 4;
        int b = 5;
        int nilai = 3 > 2 ? a : b;
        System.out.println("nilai = " + nilai);
    }
}
```

```
package guided;

public class DemoLagu {
    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-03-konsep-dasar ---
Judul : God Will Make A Way, pencipta : Don Moen
```

6. Guided 6: Constructor Overloading dan Pemanggilan Constructor dengan this()

```
package guided;

public class Buku {
    private String pengarang;
    private String judul;
    private Buku() {
        this("Rumah Kita", "GoodBles");
          // this ini digunakan untuk memanggil konstruktor yang
          // menerima dua parameter
    }
    private Buku(String judul,String pengarang)
    {
        this.judul = judul;
        this.pengarang = pengarang;
    }
    private void cetakKeLayar()
    {
```

```
        System.out.println("Judul : " + judul + " Pengarang : " + pengarang);
    }
    public static void main(String[] args)
    {
        Buku a,b ;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-03-konsep-dasar ---
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
```

C. Unguided

1. Soal Unguided 1

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode :
- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya.
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Program ini harus memuat dua jenis field: instance field (milik masing-masing object) dan class field (milik class, dipakai bersama).
- Tidak perlu inheritance, overriding, maupun abstract class.

Jawaban :

```
package unguided;

import java.util.Arrays;
import java.util.Locale;

public class PengolahSuhu {
    public static final double NILAI_KOSONG = -1.0;
    private double[] suhuHarian;

    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
```

```
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " : (kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°"
                        + "C");
            }
        }
    }

    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }
        return -1;
    }

    public void isiDataKosong() {
        int index = cariIndexKosong();
        if (index > 0 && index < suhuHarian.length - 1) {
            suhuHarian[index] = (suhuHarian[index - 1] + suhuHarian[index + 1]) /
2.0;
        }
    }

    public double hitungRataRata() {
        double total = 0;
        for (double suhu : suhuHarian) {
            total += suhu;
        }
        return total / suhuHarian.length;
    }

    public static void main(String[] args) {
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();
        System.out.println();

        int indexKosong = pengolah.cariIndexKosong();
        System.out.println("Index hari kosong (dimulai dari 0): " + indexKosong);
        System.out.println();

        pengolah.isiDataKosong();

        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
```

```
        System.out.println();

        System.out.printf(Locale.US, "Rata-rata : %.2f°C%n%n",
    pengolah.hitungRataRata());

        System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
    dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));
        System.out.println("(ikut berubah: constructor menyimpan referensi array
    yang sama)");

        assert suhuHarian[3] == 29.1 : "Imputasi data suhu hari ke-4 gagal";
        assert Math.abs(pengolah.hitungRataRata() - 29.3857) < 0.01 : "Perhitungan
    rata-rata tidak valid";
      }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ modul-03-konsep-dasar ---
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Penjelasan :

Program PengolahSuhu mengimplementasikan prinsip enkapsulasi dengan membungkus instance field suhuHarian secara private dan menetapkan penanda -1.0 sebagai class field konstan melalui public static final double NILAI_KOSONG, di

mana inisialisasi pada konstruktor memanfaatkan kata kunci this untuk membedakan field objek dari parameter lokal. Kelas ini menyediakan method tampilkanData() untuk menyajikan suhu harian sekaligus menandai nilai hilang, cariIndexKosong() untuk mendeteksi posisi data yang belum terisi (indeks 3 / Hari 4), isiDataKosong() yang menerapkan imputasi deret waktu berbasis rata-rata tetangga terdekat (suhuHarian[i-1] + suhuHarian[i+1]) / 2 hingga terisi nilai 29.1°C, serta hitungRataRata() yang menghasilkan rata-rata akhir 29.39°C. Fenomena berubahnya nilai array suhuHarian di dalam method main membuktikan sifat pass-by-reference pada objek array Java, di mana konstruktor hanya menyalin alamat memori referensi ke field objek tanpa menduplikasi isinya, sehingga modifikasi data yang dilakukan oleh method isiDataKosong() langsung tercermin pada array asli di main karena keduanya merujuk ke blok memori yang sama di Heap.

D. Kesimpulan

Pada praktikum Modul 03 ini, saya telah memahami dan berhasil mengimplementasikan pilar-pilar dasar Pemrograman Berorientasi Objek (PBO) pada Java, khususnya konsep abstraksi dan enkapsulasi (data hiding) untuk menjaga integritas data menggunakan hak akses private. Saya juga mampu membedakan secara fungsional antara instance field yang bersifat unik bagi tiap objek dan class field berbasis kata kunci static serta final untuk konstanta bersama, sekaligus memanfaatkan kata kunci this pada konstruktor dan pemanggilan internal method. Melalui studi kasus PengolahSuhu, saya memahami secara empiris mekanisme pass-by-reference pada array Java, di mana manipulasi elemen array di dalam method objek otomatis memutasi data array asli pada method main karena keduanya berbagi alamat referensi memori yang sama di Heap, membuktikan perbedaan mendasar alokasi memori antara tipe data referensi dan tipe data primitif (pass-by-value).